Exchange Matching Engine - Performance & Scalability Report

Yao-Hsuan Chang (yc658)
Ankit Raj (ar791)

Overview

This report summarizes the performance and scalability testing of our exchange matching engine. We designed the tests to simulate realistic high-load conditions and analyze how the engine behaves as concurrency increases. Our focus was on understanding the tradeoffs between thread-level parallelism and hardware (core) scaling.

Test Setup

- **Test Script**: Python-based multithreaded client sending XML orders over TCP sockets
- **Server Environment**:
  - Dockerized C++ trading engine
  - PostgreSQL database (Docker container)
  - Internal Docker network for server-DB communication
- **Metrics Collected**:
  - Throughput (TPS - transactions per second)
  - Average latency per request
- **Scaling Dimensions**:
  - Logical concurrency: by increasing the number of client threads
  - Physical concurrency: by running the engine on 1, 2, and 4 CPU cores via taskset

Results Summary

Logical Scaling (Threads on Fixed Core Set)

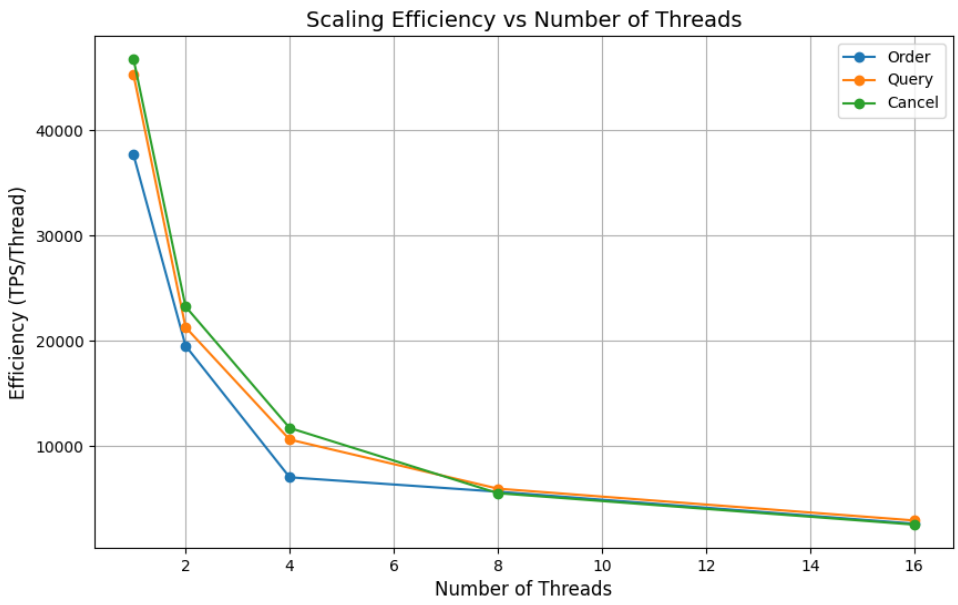
Threads	Avg TPS	Efficiency per Thread
1	37,644	100%
2	39,074	~52%
4	28,256	~19%
8	22,798	~15%
16	18,244	~7%

Hardware Scaling (Fixed Threads = 4)

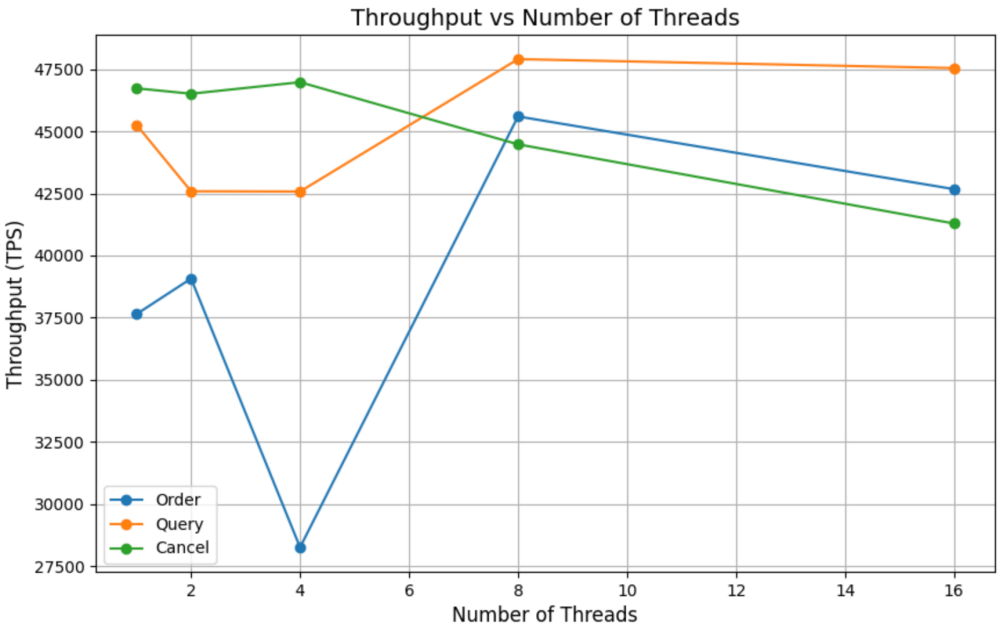
CPU Cores	Avg TPS	Observations
1	3,538	Baseline
2	7,160	Roughly 2x gain
4	11,000–12,000	Close to linear scaling

Experimental Throughput Graph

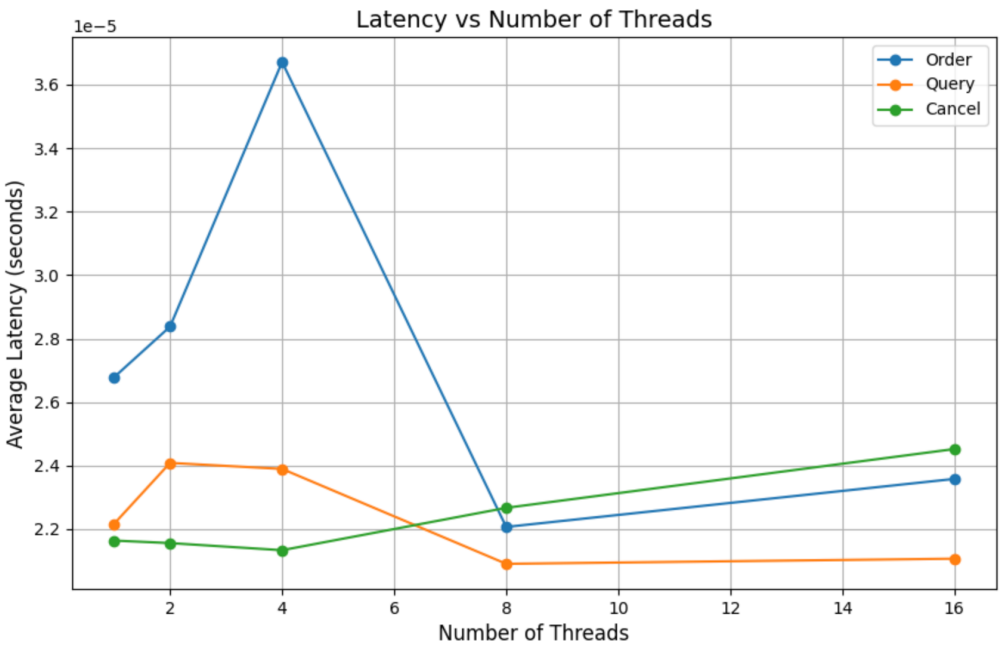
1. While increase the number of threads



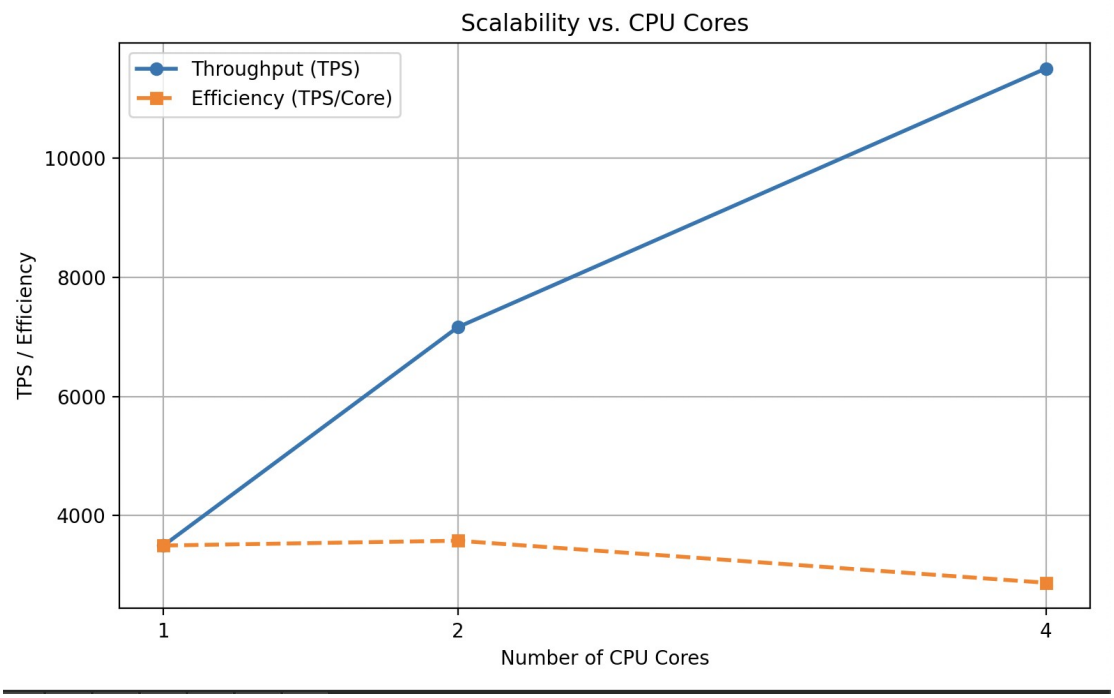
Scaling Efficiency



Throughput



Latency



Throughput while CPU Cores increase

Observations & Interpretation

- **Thread-Based Scaling**: Beyond 4 threads, performance gains taper off. This is due to internal locking, increased context switching, and limited ability to parallelize critical sections like order matching.
- **Core-Based Scaling**: When spreading the engine over more physical cores using taskset, we observed near-linear throughput improvements, confirming the design leverages multi-core hardware well.

Architectural Factors

Why Order Handling Scales Worse

Order placement involves multiple shared resources:

- Balance validation and updates
- Position adjustment
- Order book insertion
- Matching and trade execution
- DB writes

These introduce lock contention and serial dependencies.

On the other hand, QUERY and CANCEL operations:

- Are mostly read-only
- Require simpler state changes
- Hold locks for shorter durations

Locking & Logging Model

To support concurrency, our engine:

- Uses std::shared_mutex for read/write concurrency
- Employs std::thread and std::condition_variable for an async logger, reducing I/O stalls
- Pins threads to specific cores using pthread_setaffinity_np() (Linux)

Lessons Learned

- **Thread ≠ Throughput**: More threads don't always mean better performance. Lock contention is a real limiter.
- **Scale with Cores**: Spreading the engine over more physical cores (up to 4 in our case) gives consistent gains.
- **Simpler Operations Scale Better**: QUERY and CANCEL operations performed more consistently under load.
- **Profiling is Key**: We noticed ORDER operations sometimes dominate DB write time or hold locks longer.

Recommendations

1. **Sharding**: Partitioning order books by symbol can reduce contention.

2. **Fine-Grained Locks**: Lock less and lock smart. Avoid global or coarse locks.
3. **Thread Pool Tuning**: Dynamically adjust thread pool size based on core availability.
4. **Queue Management**: Cap inbound request queue depth to prevent overload during spikes.

Final Thoughts

Our team implemented core-aware logic and a modular engine to maximize scalability. While logical concurrency hits diminishing returns due to internal synchronization, the system scales well with physical hardware which is a **Classic example of Amdahl's Law**. These results give us a solid baseline for further optimization and future production deployment.

This analysis, backed by both raw throughput data and architectural reasoning, provides a realistic picture of where our exchange matching engine performs well and where it needs refinement.